Tomsk State University

Laboratory Report

Import data to neo4j database

Student: Liu Ye

Group: 932530

Date: October 1, 2026

Contents

1 Import Data into Neo4j Database **2**

1.1 Copy Dataset Files into Neo4j Container 2
1.2 Import House Nodes 2
1.3 Checking House Nodes 3
1.4 Creating Character-House Relationships 4
1.5 Query Character-House Relationship 5
1.6 Creating Family Relationships 5
1.7 Query Family Relationships 6

2 Importing Additional Kaggle Dataset **7**

2.1 Dataset Selection and Graph Modeling 7
2.2 Preparing Dataset Files 8
2.3 Creating Olist Graph Nodes 8
2.4 Creating Customer-Order Relationship 10
2.5 Creating Order-Product Relationship 11
2.6 Final Olist Graph Visualization 12

3 Conclusion **13**

1 Import Data into Neo4j Database

1.1 Copy Dataset Files into Neo4j Container

The first step is to copy all CSV files into the Neo4j import directory. The commands shown in Figure 0 were executed in the Linux terminal.

```
sudo docker cp overview.csv neo4j:/import/
sudo docker cp characters.csv neo4j:/import/
sudo docker cp characters_episodes.csv neo4j:/import/
sudo docker cp houses.csv neo4j:/import/
sudo docker cp characters_houses.csv neo4j:/import/
sudo docker cp family_ties.csv neo4j:/import/
```

```
(base) liuye@debian:~/下载$ sudo docker cp overview.csv neo4j:/import/
Successfully copied 1.5kB (transferred 3.07kB) to neo4j:/import/
(base) liuye@debian:~/下载$ sudo docker cp characters.csv neo4j:/import/
Successfully copied 23.9kB (transferred 25.6kB) to neo4j:/import/
(base) liuye@debian:~/下载$ sudo docker cp characters_episodes.csv neo4j:/import/
Successfully copied 60.6kB (transferred 62.5kB) to neo4j:/import/
(base) liuye@debian:~/下载$ sudo docker cp houses.csv neo4j:/import/
Successfully copied 2.59kB (transferred 4.61kB) to neo4j:/import/
(base) liuye@debian:~/下载$ sudo docker cp characters_houses.csv neo4j:/import/
Successfully copied 13.7kB (transferred 15.4kB) to neo4j:/import/
(base) liuye@debian:~/下载$ sudo docker cp family_ties.csv neo4j:/import/
Successfully copied 8.75kB (transferred 10.8kB) to neo4j:/import/
(base) liuye@debian:~/下载$ sudo docker exec neo4j ls -lh /import/
total 120K
-rw-rw-r-- 1 1000 1000  24K Oct  1 04:17 characters.csv
-rw-rw-r-- 1 1000 1000  60K Oct  1 04:18 characters_episodes.csv
-rw-rw-r-- 1 1000 1000  14K Oct  1 09:22 characters_houses.csv
-rw-rw-r-- 1 1000 1000 8.6K Oct  1 10:06 family_ties.csv
-rw-rw-r-- 1 1000 1000 2.6K Oct  1 04:18 houses.csv
-rw-rw-r-- 1 1000 1000 1.5K Oct  1 04:17 overview.csv
(base) liuye@debian:~/下载$
```

Figure 1: Copying CSV files into Neo4j container

The result shows that the files were successfully copied into the Neo4j container. These files are required by the LOAD CSV command later.

1.2 Import House Nodes

The first Cypher command creates house nodes from houses.csv.

```
LOAD CSV WITH HEADERS FROM "file:///houses.csv" AS H
CREATE (houses:Houses {
    Link: H.Link,
    Name: H.Name
});
```

The command uses LOAD CSV WITH HEADERS to read every row of the CSV file. Each row becomes a node with label Houses. The Link property stores the unique identifier and Name stores the house name.

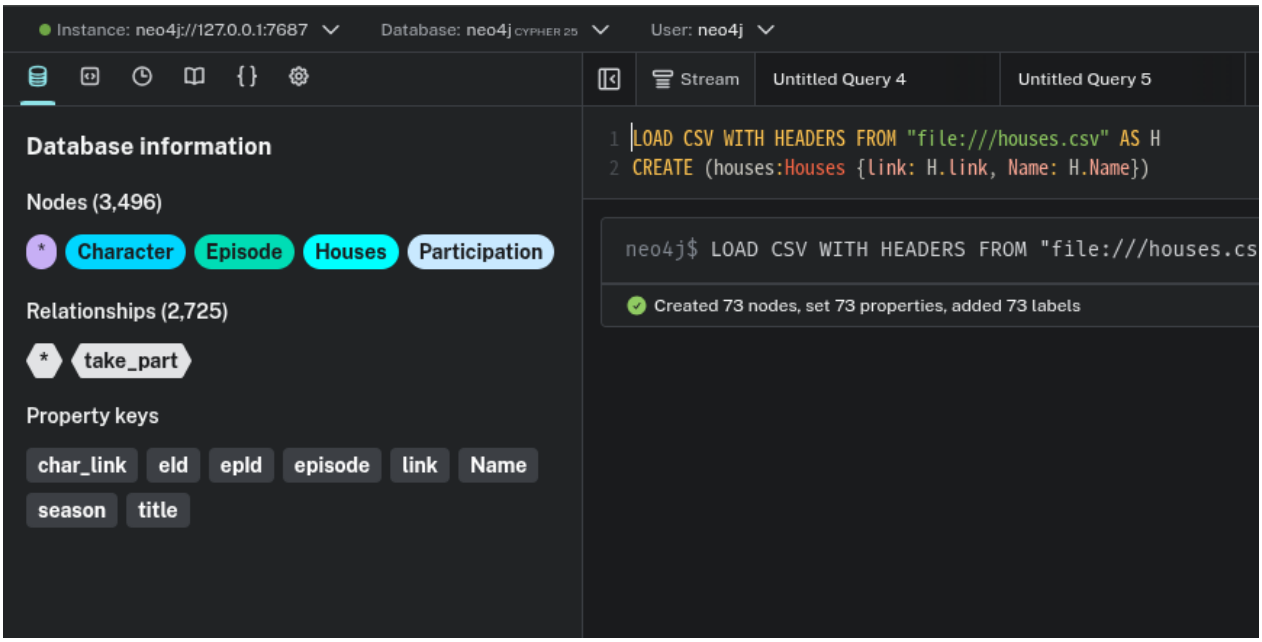


Figure 2: Creating Houses nodes

The execution result shows that the Houses nodes were created successfully. The database now contains Character, Episode and Houses node types.

1.3 Checking House Nodes

The following query checks the imported house nodes.

```
MATCH (n:Houses)
RETURN n
LIMIT 21;
```

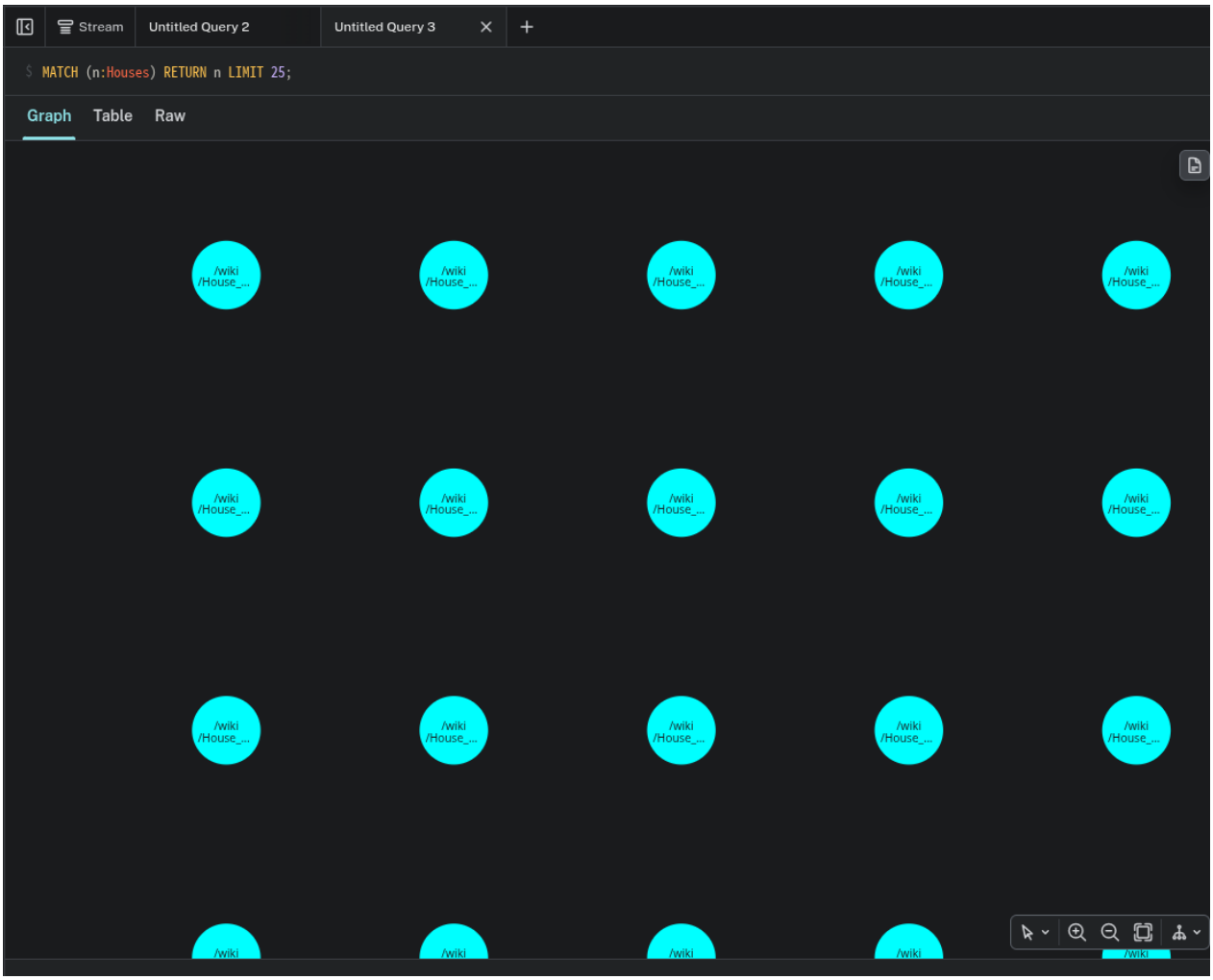


Figure 3: Displaying imported house nodes

The result displays several Houses nodes, proving that the CSV import operation was successful.

1.4 Creating Character-House Relationships

The relationship between characters and houses was created using characters_houses.csv.

```
LOAD CSV WITH HEADERS FROM "file:///characters_houses.csv" AS CH

MATCH (c:Character {Link: CH.character}),
(h:Houses {Link: CH.house})

CREATE (c)-[:BELONGS_TO]->(h);
```

This query matches existing Character nodes and Houses nodes by their Link property and creates BELONGS_TO relationships.

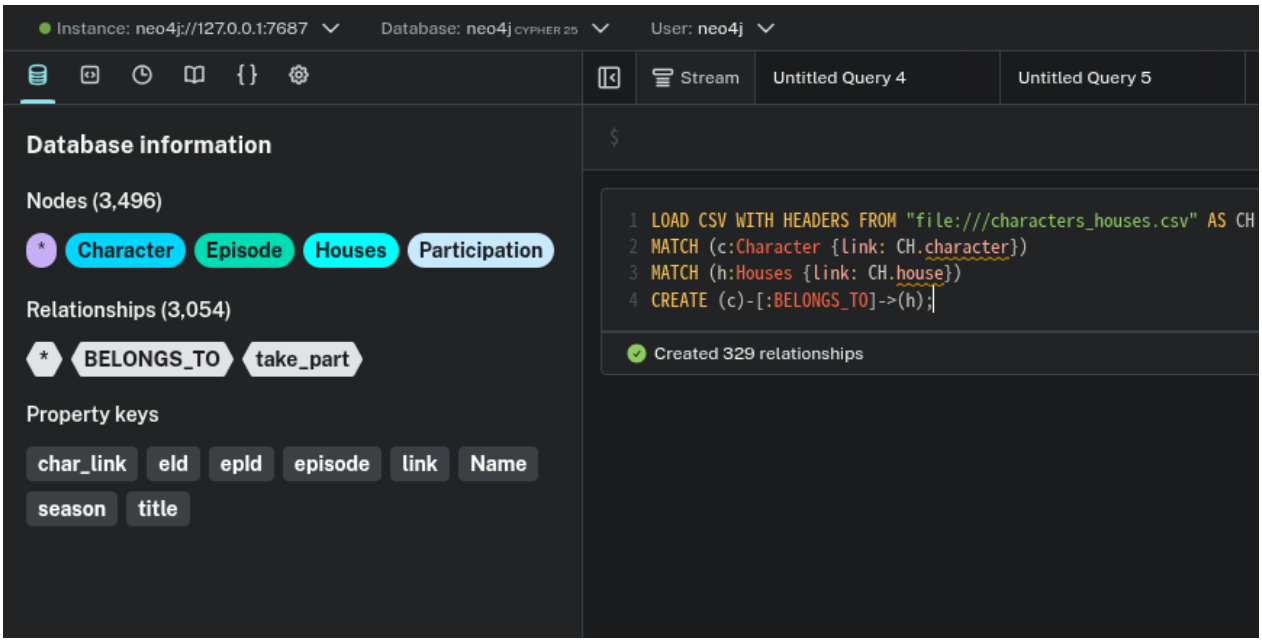


Figure 4: Creating BELONGS_TO relationships

1.5 Query Character-House Relationship

```cypher
MATCH (c:Character)-[r:BELONGS_TO]->(h:Houses)
WHERE c.Name = 'Razdal mo Eraz'
    OR c.Name = "Second Son"
RETURN c,r,h;
```

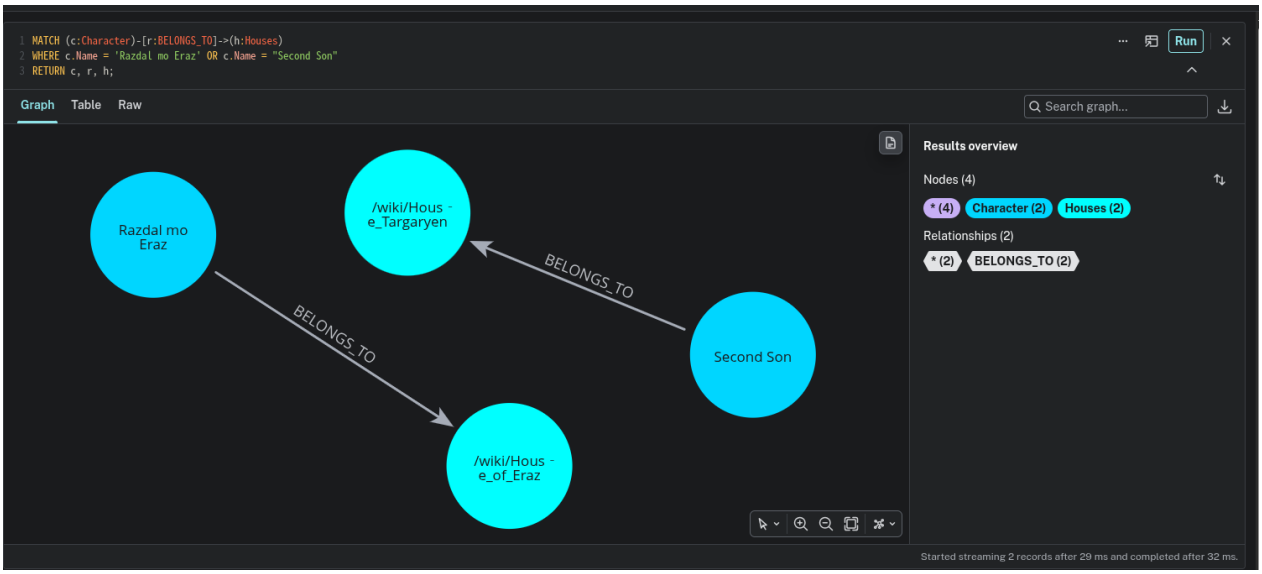


Figure 5: BELONG_TO query result

The returned graph contains Character nodes, Houses nodes and BELONGS_TO edges. This confirms that characters are correctly connected to their houses.

1.6 Creating Family Relationships

Family relationships were imported from family_ties.csv.

```
LOAD CSV WITH HEADERS FROM "file:///family_ties.csv" AS F

MATCH (c1:Character {Link:F.character1}),
(c2:Character {Link:F.character2})

CREATE (c1)-[:FAMILY {
    relationship:F.relationship
}]->(c2);
```

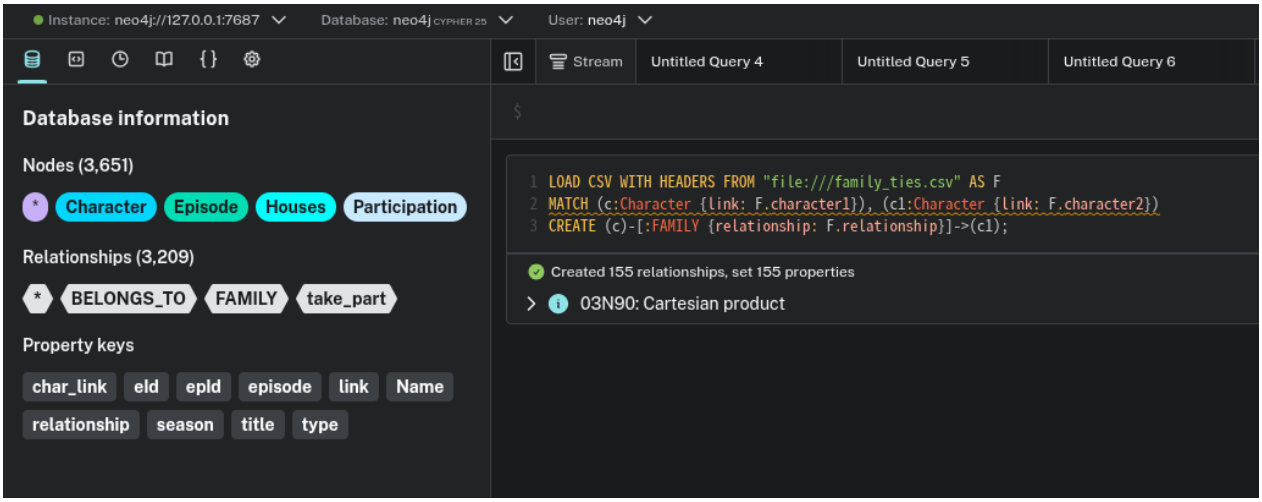


Figure 6: Importing family relationships

The query creates FAMILY edges between two Character nodes and stores the relationship type as a property.

1.7 Query Family Relationships

```
MATCH (c:Character)-[r:FAMILY]->(p:Character)

WHERE c.Name = "Waymar Royce"

RETURN c,r,p;
```

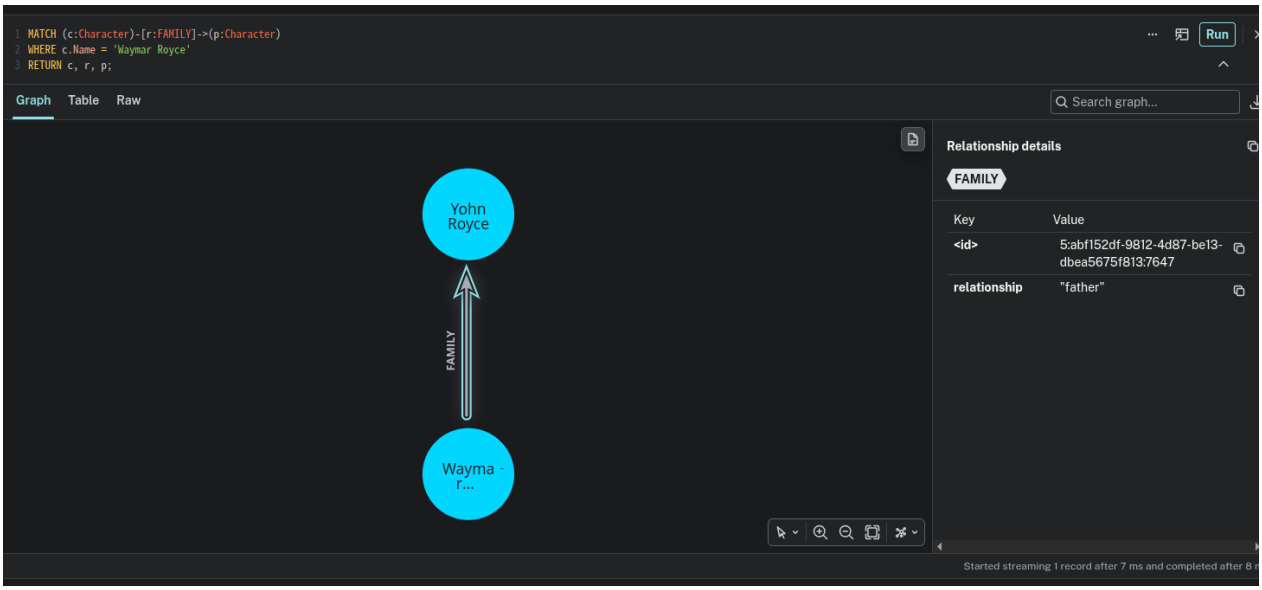


Figure 7: Family relationship query result

The result shows that Waymar Royce is connected with another character through a FAMILY relationship. The relationship property describes the family type.

2 Importing Additional Kaggle Dataset

2.1 Dataset Selection and Graph Modeling

According to the laboratory requirement, a dataset from Kaggle that can be represented as a graph was selected and imported into Neo4j.

The selected dataset is the Olist Brazilian E-Commerce dataset. This dataset contains information about customers, orders, products, and transactions. Because the dataset contains clear entity relationships, it can be naturally represented as a graph database.

The graph model was designed as follows:

- Customer: represents users who place orders.
- Order: represents purchase transactions.
- Product: represents purchased products.

The relationships between nodes are:

- Customer –PLACED–> Order
- Order –CONTAINS–> Product

This graph structure allows Neo4j to directly represent the purchasing relationships between customers and products.

2.2 Preparing Dataset Files

Before importing data, the CSV files were copied into the Neo4j import folder.

The following commands were executed:

```
sudo docker cp olist_customers_dataset.csv neo4j:/import/
sudo docker cp olist_orders_dataset.csv neo4j:/import/
sudo docker cp olist_products_dataset.csv neo4j:/import/
sudo docker cp olist_order_items_dataset.csv neo4j:/import/
```

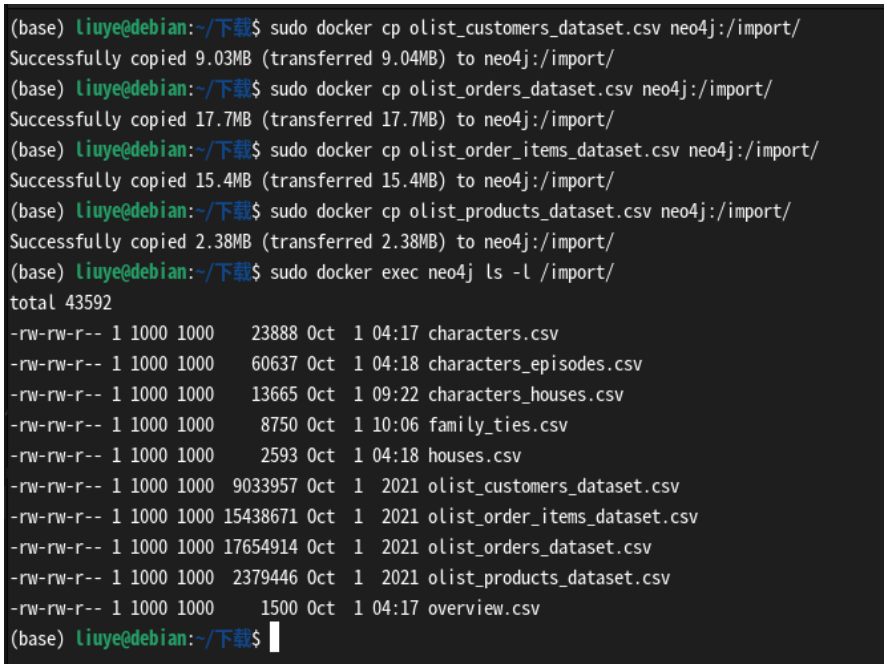


Figure 8: Copying Olist CSV files into Neo4j container

The execution result shows that all required CSV files were successfully copied into the Neo4j container.

These files contain different types of information:

- olist_customers_dataset.csv stores customer information.
- olist_orders_dataset.csv stores order information.
- olist_products_dataset.csv stores product information.
- olist_order_items_dataset.csv stores the relationship between orders and products.

2.3 Creating Olist Graph Nodes

After importing the Olist dataset, three types of nodes were created: Customer, Order and Product.

Instead of creating each node type separately, all three node types were created together using Cypher commands.

The Cypher code is shown below:

```
LOAD CSV WITH HEADERS
FROM "file:///olist_customers_dataset.csv" AS C
CREATE (customer:Customer{
    customer_id:C.customer_id,
    city:C.customer_city,
    state:C.customer_state
});
LOAD CSV WITH HEADERS
FROM "file:///olist_orders_dataset.csv" AS O

CREATE (order:Order{
    order_id:O.order_id,
    status:O.order_status,
    purchase_date:O.order_purchase_timestamp
});
LOAD CSV WITH HEADERS
FROM "file:///olist_products_dataset.csv" AS P

CREATE (product:Product{
    product_id:P.product_id,
    category:P.product_category_name
});
```

The above Cypher commands import three different CSV files and transform the relational data into graph nodes.

Three node labels are created:

- Customer node: stores customer information including customer ID, city and state.
- Order node: stores order information including order ID, order status and purchase date.
- Product node: stores product information including product ID and category.

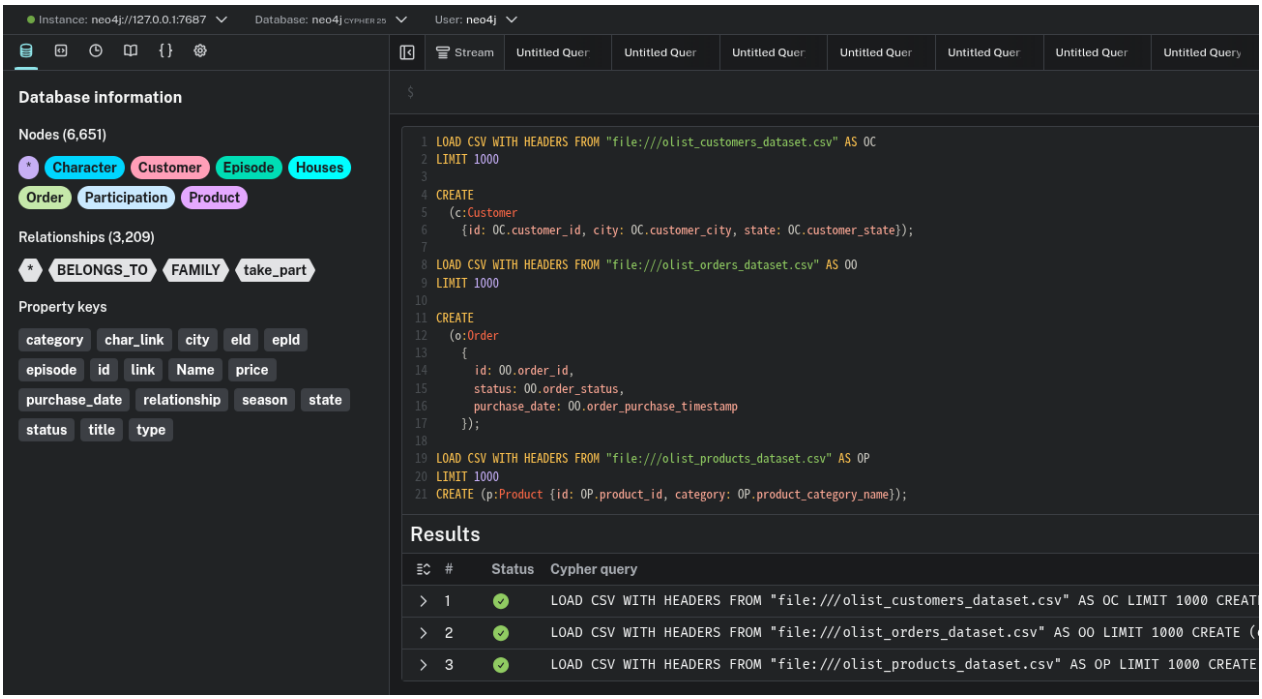


Figure 9: Creating Customer, Order and Product nodes

The execution result shows that the three types of nodes were successfully created in Neo4j.

2.4 Creating Customer-Order Relationship

After creating the nodes, relationships were added between different entities.

The first relationship connects customers and orders.

The relationship represents that a customer placed an order.

The Cypher code is:

```
LOAD CSV WITH HEADERS
FROM "file:///olist_orders_dataset.csv" AS O
MATCH (c:Customer{
    customer_id:O.customer_id
})
MATCH (o:Order{
    order_id:O.order_id
})
CREATE (c)-[:PLACED]->(o);
```

The MATCH statement searches for the corresponding Customer and Order nodes.

The CREATE statement creates a directed relationship:

$$\text{Customer} \rightarrow \text{Order}$$

The relationship type is PLACED, which indicates that a customer created an order.

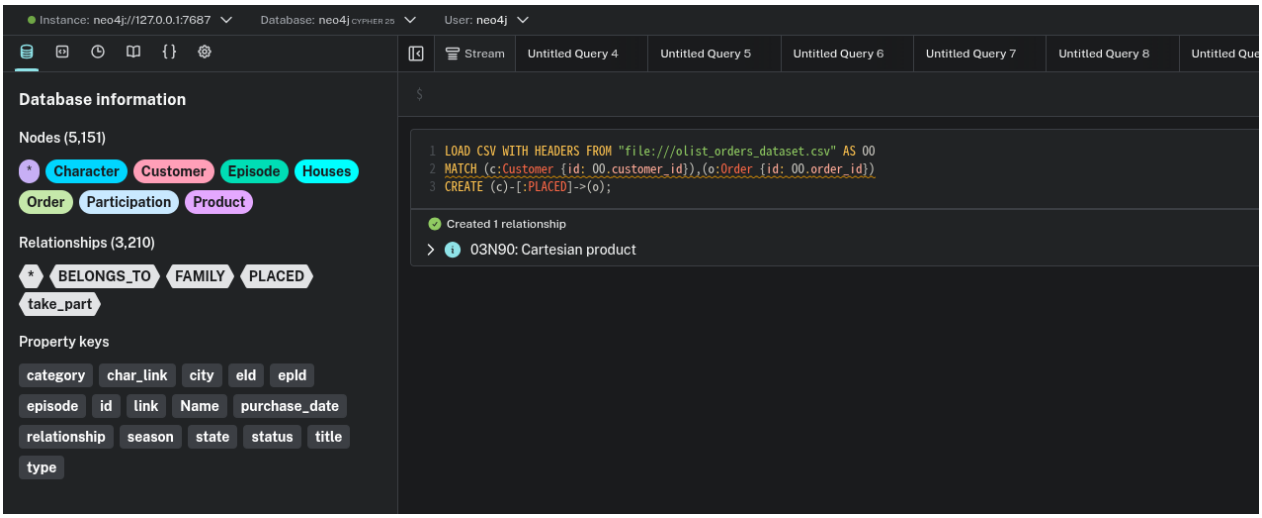


Figure 10: Creating Customer-Order relationship

The execution result shows that Customer nodes are successfully connected with their corresponding Order nodes.

2.5 Creating Order-Product Relationship

The second relationship connects orders with products.

This relationship describes which products are included in each order.

The Cypher code is:

```
LOAD CSV WITH HEADERS
FROM "file:///olist_order_items_dataset.csv" AS I
MATCH (o:Order{
    order_id:I.order_id
})
MATCH (p:Product{
    product_id:I.product_id
})
CREATE (o)-[:CONTAINS {price:I.price}]->(p);
```

The CONTAINS relationship contains an additional property:

- price: the product price in this order

The graph structure becomes:

$$\text{Customer} \rightarrow \text{Order} \rightarrow \text{Product}$$

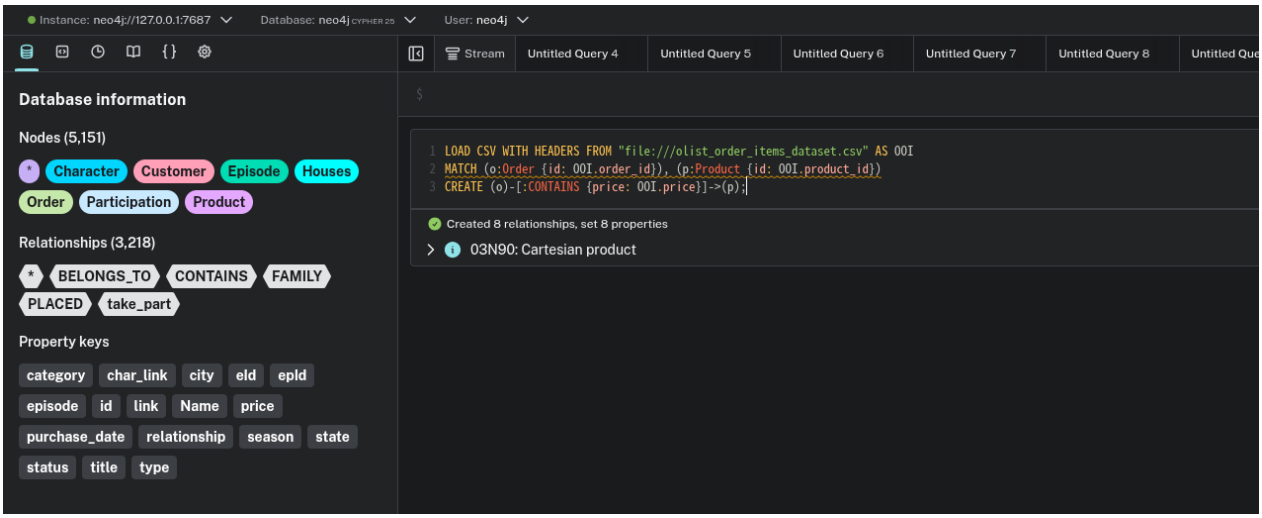


Figure 11: Creating Order-Product relationship

The result shows that orders are connected with products successfully. The graph now contains both customer purchasing behavior and product information.

2.6 Final Olist Graph Visualization

After creating all nodes and relationships, a final Cypher query was executed to display the complete Olist graph structure in Neo4j.

The Cypher query is:

```
MATCH (c:Customer)
      -[:PLACED]->(o:Order)
      -[:CONTAINS]->(p:Product)
RETURN c,o,p
LIMIT 20;
```

The MATCH clause searches for connected paths in the graph database.

The query starts from Customer nodes and follows the PLACED relationship to find corresponding Order nodes. Then, through the CONTAINS relationship, the related Product nodes are retrieved.

The query returns three connected node types:

- Customer: the user who placed the order
- Order: the transaction record
- Product: the purchased item

The LIMIT keyword restricts the returned results to 20 paths, which prevents Neo4j from displaying too many nodes and makes the graph visualization easier to observe.

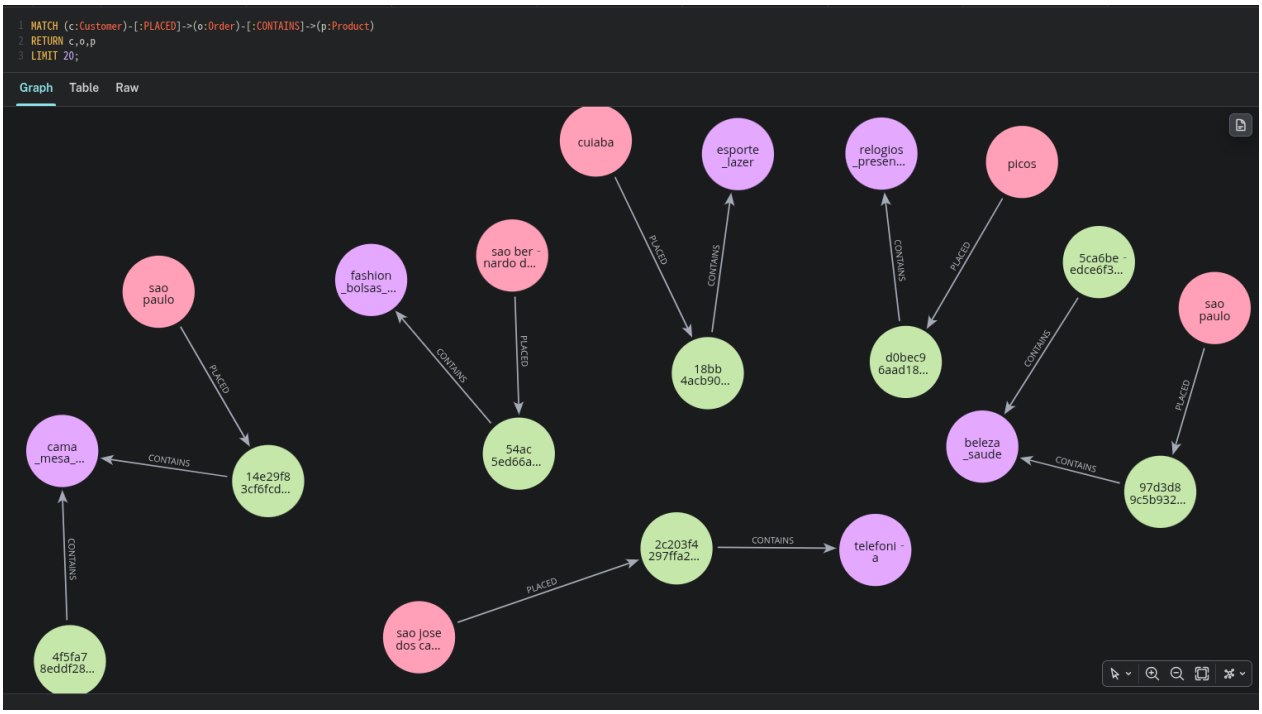


Figure 12: Final Olist graph database result

The final graph contains three node types:

- Customer
- Order
- Product

and two relationship types:

- PLACED: Customer to Order
- CONTAINS: Order to Product

The execution result shows that Customer nodes are connected with Order nodes through the PLACED relationship, and Order nodes are connected with Product nodes through the CONTAINS relationship.

This demonstrates that the Olist dataset was successfully transformed from traditional CSV tables into a graph database model in Neo4j.

3 Conclusion

The experiment successfully imported multiple CSV datasets into Neo4j. The Cypher LOAD CSV command was used to create nodes and relationships. The Game of Thrones dataset was extended with house information and family relationships. The Olist dataset was also transformed into a graph structure.